

Activités d'Intégration de Compétences —

Partie 5

Objectif général

Introduire deux piliers fondamentaux des architectures micro services modernes : la **communication asynchrone** entre services via un broker de messages (Apache Kafka), et l'**observabilité** de la plateforme via des logs centralisés, un monitoring des métriques et des health checks. À l'issue de cette partie, votre architecture sera non seulement fonctionnelle, mais aussi observable et résiliente.

Q1 — Créer une branche Git `version-5` et ajouter le Sprint 5 dans Jira

Indication : Créez la branche `version-5` à partir de `version-4` et ouvrez un Sprint 5 dans votre projet Jira. Prenez le temps de rédiger les User Stories avant de commencer : par exemple *"En tant qu'administrateur, je veux recevoir une notification automatique chaque fois qu'un étudiant est ajouté", ou encore "En tant qu'exploitant, je veux consulter les logs centralisés de tous les micro services depuis une interface unique"*. Ces stories ancrent les développements techniques dans un besoin utilisateur concret, ce qui est l'essence du Scrum.

```
git checkout version-4
git checkout -b version-5
```

Q2 — Ajouter le micro service `notification-service` avec Apache Kafka

Indication : Avant d'écrire la moindre ligne de code, il est essentiel de comprendre *pourquoi* Kafka plutôt qu'un appel HTTP direct. Dans l'architecture actuelle, si `etudiant-service` appelait `notification-service` via Feign lors de l'ajout d'un étudiant, les deux services seraient **couplés temporellement** : si `notification-service` est en panne, la création de l'étudiant échouerait aussi. Kafka résout ce problème en agissant comme un intermédiaire durable : `etudiant-service` publie un événement dans un **topic** Kafka et continue son chemin sans attendre. `notification-service` consomme cet événement quand il est disponible, de manière totalement indépendante. C'est le principe de la **communication asynchrone** et du **découplage fort** entre services.

Architecture du flux de messages

L'architecture que vous allez implémenter suit ce flux précis : lorsqu'un étudiant est créé ou qu'une note est enregistrée, le service producteur publie un message JSON dans un topic Kafka dédié. Le `notification-service` est abonné à ces topics et simule l'envoi d'un email en loggant

le message reçu. Dans un contexte réel, vous remplacerez ce log par un appel à un service d'emailing (SendGrid, JavaMailSender, etc.).

```
etudiant-service → topic: etudiant-created → notification-service
grading-service  → topic: note-created      → notification-service
```

Configuration Kafka dans `etudiant-service` (Producteur)

Ajoutez la dépendance Spring Kafka dans votre `pom.xml` :

```
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

Configurez le producteur dans `application.yml` :

```
spring:
  kafka:
    bootstrap-servers: kafka:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
```

Créez un `KafkaProducerService` dans `etudiant-service` qui publie un événement après chaque création d'étudiant. Remarquez que ce service ne connaît pas `notification-service` : il parle uniquement à Kafka, ce qui matérialise le découplage.

```
// etudiant-service/src/main/java/.../kafka/KafkaProducerService.java
@Service
@RequiredArgsConstructor
public class KafkaProducerService {

    private final KafkaTemplate<String, EtudiantEvent> kafkaTemplate;

    // On publie l'événement sur le topic "etudiant-created".
    // Le service qui crée l'étudiant appelle cette méthode sans se soucier
    // de qui va traiter l'événement ni quand.
    public void publishEtudiantCreated(EtudiantDTO etudiant) {
        EtudiantEvent event = EtudiantEvent.builder()
            .etudiantId(etudiant.getId())
            .nom(etudiant.getNom())
            .email(etudiant.getEmail())
            .timestamp(LocalDateTime.now())
            .build();
        kafkaTemplate.send("etudiant-created", event);
    }
}
```

Appelez cette méthode depuis `EtudiantService.save()` juste après la persistance :

```
// Dans EtudiantService.java
public EtudiantDTO save(EtudiantDTO dto) {
    Etudiant saved = repository.save(mapper.toEntity(dto));
    // On publie l'événement de manière asynchrone.
    // Même si Kafka est temporairement indisponible, la transaction JPA
    // est déjà committée – les deux opérations sont indépendantes.
    kafkaProducerService.publishEtudiantCreated(mapper.toDto(saved));
    return mapper.toDto(saved);
}
```

Création du notification-service (Consommateur)

Créez un nouveau projet Spring Boot `notification-service` dans un dossier dédié. Ce service n'a pas besoin de base de données pour cet exercice : son seul rôle est d'écouter les topics Kafka et de simuler l'envoi de notifications. Configurez-le comme consommateur Kafka :

```
# notification-service/src/main/resources/application.yml
spring:
  kafka:
    bootstrap-servers: kafka:9092
    consumer:
      group-id: notification-group
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer:
        org.springframework.kafka.support.serializer.JsonDeserializer
      properties:
        spring.json.trusted.packages: ""
```

Créez le listener qui consomme les événements. L'annotation `@KafkaListener` fait tout le travail d'abonnement : Spring s'occupe de la connexion au broker, de la désérialisation JSON et de l'appel de votre méthode à chaque message reçu.

```
// notification-service/src/main/java/.../listener/NotificationListener.java
@Component
@Slf4j // Lombok pour le logger
public class NotificationListener {

    // Ce topic correspond exactement au nom utilisé dans le producteur.
    // Spring Kafka crée automatiquement le consumer group et gère les offsets.
    @KafkaListener(topics = "etudiant-created", groupId = "notification-group")
    public void onEtudiantCreated(EtudiantEvent event) {
        log.info("📧 [NOTIFICATION] Nouvel étudiant inscrit : {} (ID: {}). Email de bienvenue simulé envoyé à {}",
            event.getNom(), event.getEtudiantId(), event.getEmail());
    }

    @KafkaListener(topics = "note-created", groupId = "notification-group")
    public void onNoteCreated(NoteEvent event) {
        log.info("📧 [NOTIFICATION] Nouvelle note enregistrée pour l'étudiant ID {} - Matière : {}, Valeur : {}",
            event.getStudentId(), event.getMatiere(), event.getValeur());
    }
}
```


Mise à jour du `docker-compose.yml`

Kafka nécessite **Zookeeper** pour la coordination du cluster (dans les versions antérieures à Kraft). Ajoutez les deux conteneurs ainsi que le `notification-service` :

```
zookeeper:
  image: confluentinc/cp-zookeeper:7.5.0
  environment:
    ZOOKEEPER_CLIENT_PORT: 2181

kafka:
  image: confluentinc/cp-kafka:7.5.0
  depends_on:
    - zookeeper
  ports:
    - "9092:9092"
  environment:
    KAFKA_BROKER_ID: 1
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    # PLAINTEXT_HOST permet aux outils tournant sur votre machine (hors Docker)
    # d'accéder à Kafka via localhost:9092.
    KAFKA_ADVERTISED_LISTENERS:
PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:29092
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

notification-service:
  build: ./notification-service
  depends_on:
    - kafka
  environment:
    SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:9092
```

Q3 — Ajouter une base minimale d'observabilité

Indication : L'observabilité répond à une question fondamentale : *"Que se passe-t-il réellement dans mon système en production ?"* Sans elle, un bug ou une dégradation de performance peut passer inaperçu pendant des heures. L'observabilité repose classiquement sur trois piliers — les **logs**, les **métriques** et les **traces** — souvent appelés les "trois piliers de l'observabilité". Dans cette partie, vous mettez en place les deux premiers ainsi que les health checks, ce qui constitue une base solide et suffisante pour un projet de Master.

Logs centralisés avec la stack ELK (Elasticsearch + Logstash + Kibana)

En l'absence de centralisation, les logs sont éparpillés dans chaque conteneur Docker et vous devez faire `docker logs <container>` pour chaque service séparément. La stack ELK résout ce problème : tous les micro services envoient leurs logs à **Logstash** qui les transforme et les indexe dans **Elasticsearch**. **Kibana** offre ensuite une interface graphique pour les rechercher, les filtrer et les visualiser.

Commencez par configurer chaque micro service Spring Boot pour écrire ses logs au format JSON structuré, ce qui facilite l'indexation dans Elasticsearch. Ajoutez `logstash-logback-encoder` dans chaque `pom.xml` :

```
<dependency>
  <groupId>net.logstash.logback</groupId>
  <artifactId>logstash-logback-encoder</artifactId>
  <version>7.4</version>
</dependency>
```

Créez un fichier `logback-spring.xml` dans `src/main/resources/` de chaque service pour définir deux appenders : un pour la console (lisible par un humain pendant le développement) et un pour Logstash (JSON structuré pour la centralisation) :

```
<!-- src/main/resources/logback-spring.xml -->
<configuration>
  <!-- Appender console pour le développement local -->
  <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>%d{HH:mm:ss} %-5level [%thread] %logger{36} - %msg%n</pattern>
    </encoder>
  </appender>

  <!-- Appender Logstash pour la centralisation en production -->
  <appender name="LOGSTASH"
    class="net.logstash.logback.appender.LogstashTcpSocketAppender">
    <destination>logstash:5000</destination>
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
      <!-- Ce champ "service" permet de filtrer par micro service dans Kibana -->
      <customFields>{"service":"etudiant-service"}</customFields>
    </encoder>
  </appender>

  <root level="INFO">
    <appender-ref ref="CONSOLE"/>
    <appender-ref ref="LOGSTASH"/>
  </root>
</configuration>
```

Ajoutez les trois composants ELK dans votre `docker-compose.yml` :

```

elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:8.12.0
  environment:
    - discovery.type=single-node
    - xpack.security.enabled=false # Désactivé pour simplifier en dev
  ports:
    - "9200:9200"

logstash:
  image: docker.elastic.co/logstash/logstash:8.12.0
  ports:
    - "5000:5000" # Port TCP pour la réception des logs
    - "9600:9600" # Port de monitoring Logstash
  volumes:
    - ./observability/logstash/pipeline:/usr/share/logstash/pipeline
  depends_on:
    - elasticsearch

kibana:
  image: docker.elastic.co/kibana/kibana:8.12.0
  ports:
    - "5601:5601"
  depends_on:
    - elasticsearch

```

Créez le pipeline Logstash dans `observability/logstash/pipeline/logstash.conf` :

```

# Ce fichier définit comment Logstash reçoit, traite et stocke les logs.
# input : écoute les logs JSON envoyés par les services Spring Boot sur le port TCP
5000.
# filter : décode le JSON automatiquement (déjà géré par LogstashEncoder).
# output : indexe chaque log dans Elasticsearch avec un index horodaté.
input {
  tcp {
    port => 5000
    codec => json_lines
  }
}

output {
  elasticsearch {
    hosts => ["elasticsearch:9200"]
    index => "microservices-logs-%{+YYYY.MM.dd}"
  }
}

```

Une fois la stack démarrée, accédez à Kibana sur `http://localhost:5601` , créez un index pattern `microservices-logs-*` et explorez vos logs depuis le menu **Discover**. Vous pourrez filtrer par champ `service` pour isoler les logs d'un micro service spécifique.

Monitoring des métriques avec Spring Actuator + Prometheus + Grafana

Spring Boot Actuator expose des métriques sur votre application (mémoire JVM, nombre de requêtes HTTP, latences, connexions à la base de données, etc.).

Prometheus collecte ces métriques à intervalles réguliers en interrogeant un endpoint dédié (`/actuator/prometheus`). **Grafana** se connecte à Prometheus et vous permet de créer des tableaux de bord visuels interactifs. Ces trois outils forment un trio complémentaire très répandu dans l'industrie.

Ajoutez les dépendances Actuator et Micrometer dans chaque service Spring Boot :

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<!-- Micrometer est le pont entre Spring et Prometheus -->
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

Configurez Actuator dans `application.yml` pour exposer les endpoints nécessaires. Attention : n'exposez jamais `shutdown` en production sans authentification.

```
management:
  endpoints:
    web:
      exposure:
        # On expose health, info et prometheus. En production, restreignez davantage.
        include: health, info, prometheus, metrics
    endpoint:
      health:
        # "always" affiche les détails même si le service est dégradé – utile pour le
        debug.
        show-details: always
  metrics:
    tags:
      # Ce tag Micrometer ajoute le nom du service à chaque métrique,
      # ce qui permet de les distinguer dans Prometheus/Grafana.
      application: ${spring.application.name}
```

Créez la configuration Prometheus dans `observability/prometheus/prometheus.yml` . Prometheus va interroger le endpoint `/actuator/prometheus` de chaque service toutes les 15 secondes pour collecter les métriques :

```

global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'etudiant-service'
    static_configs:
      - targets: ['etudiant-service:8081']
    metrics_path: '/actuator/prometheus'

  - job_name: 'grading-service'
    static_configs:
      - targets: ['grading-service:8082']
    metrics_path: '/actuator/prometheus'

  - job_name: 'notification-service'
    static_configs:
      - targets: ['notification-service:8083']
    metrics_path: '/actuator/prometheus'

```

Ajoutez Prometheus et Grafana dans le `docker-compose.yml` :

```

prometheus:
  image: prom/prometheus:v2.50.0
  ports:
    - "9090:9090"
  volumes:
    - ./observability/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml

grafana:
  image: grafana/grafana:10.3.0
  ports:
    - "3001:3000"
  depends_on:
    - prometheus
  environment:
    # Identifiants par défaut pour le développement uniquement.
    GF_SECURITY_ADMIN_USER: admin
    GF_SECURITY_ADMIN_PASSWORD: admin

```

Dans Grafana (`http://localhost:3001`), ajoutez Prometheus comme data source (`http://prometheus:9090`), puis importez le dashboard communautaire **Spring Boot Statistics** (ID `6756` ou `12900` sur grafana.com) pour obtenir immédiatement des visualisations préconstruites des métriques JVM et HTTP.

Health Checks

Indication : Spring Boot Actuator expose automatiquement un endpoint

/actuator/health qui agrège l'état de tous les composants de votre application (base de données, Redis, Kafka, etc.). Configurez Docker Compose pour utiliser ces health checks afin que les services dépendants attendent réellement que leurs dépendances soient *prêtes* (et pas seulement *démarrées*). La subtilité est importante : `depends_on` sans health check garantit seulement que le conteneur a démarré, pas que l'application à l'intérieur est prête à recevoir des requêtes.

```
# Exemple de health check Docker Compose pour etudiant-service
etudiant-service:
  build: ./api-spring-boot
  depends_on:
    postgres:
      condition: service_healthy # Attend que postgres soit healthy
    kafka:
      condition: service_started # Kafka n'a pas de health check standard
  healthcheck:
    # Spring Actuator répond 200 si l'application et ses dépendances sont saines.
    test: ["CMD", "curl", "-f", "http://localhost:8081/actuator/health"]
    interval: 30s
    timeout: 10s
    retries: 5
    start_period: 60s # Délai initial pour laisser la JVM démarrer

postgres:
  image: postgres:15
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 10s
    timeout: 5s
    retries: 5
```

Structure du dépôt GitHub attendue

```
/projet-etudiants/
├── api-spring-boot/
│   ├── src/main/resources/
│   │   └── logback-spring.xml          # Configuration logs JSON + Logstash
├── grading-service/
│   ├── src/main/resources/
│   │   └── logback-spring.xml
├── notification-service/                # Nouveau micro service Kafka consumer
│   ├── src/main/java/.../
│   │   ├── listener/
│   │   │   └── NotificationListener.java
│   │   └── event/
│   │       ├── EtudiantEvent.java
│   │       └── NoteEvent.java
│   ├── src/main/resources/
│   │   ├── application.yml
│   │   └── logback-spring.xml
│   └── pom.xml
├── observability/
│   ├── logstash/
│   │   └── pipeline/
│   │       └── logstash.conf          # Pipeline de traitement des logs
│   └── prometheus/
│       └── prometheus.yml             # Configuration du scraping
└── docker-compose.yml                 # Mis à jour avec ELK, Prometheus, Grafana,
Kafka
```

💡 **Pour aller plus loin** : les logs et les métriques couvrent deux des trois piliers de l'observabilité. Le troisième pilier — le **tracing distribué** — permet de suivre une requête à travers plusieurs micro services en lui assignant un identifiant unique (trace ID). Des outils comme **Zipkin** ou **Jaeger** avec Spring Cloud Sleuth permettent de visualiser ces traces sous forme de graphes de dépendances temporelles. C'est la fonctionnalité qui fait vraiment la différence quand il s'agit de diagnostiquer une lenteur dans une chaîne de micro services.